

---

# Lab D: Assembly Language "Do at Home" Lab

This lab continues the assembly-language work from Lab 3 and focuses on calling C standard-library functions from assembly, dynamic allocation, multi-precision arithmetic, and pseudo-random number generation.

**This lab may be done in pairs.**

As usual, you should read and understand the reading material and complete Part 0 before attempting the lab assignment.

**Important:** For this lab, unlike Lab 3, you are supposed to use `stdlib` functions. Make sure you compile and link with the CDECL conventions; otherwise, the C-to-assembly interface you have used before will not work.

## Lab Goals

- Assembly language primer: improving proficiency in assembly language features.
- Interfacing C to assembly code, continued.
- Using dynamically allocated memory.
- Multi-precision addition.
- Pseudo-random number generation.

---

## Part 0: Basic Command-Line Arguments Printing Using `stdlib`

Part 0 is crucial for the successful completion of this lab. Make sure you finish it and understand it before implementing your submitted program.

---

Read the Assembly lecture **Assembly Language Primer**. For this task you must understand the arguments of `main()`, how to access the arguments of a function in assembly language, and how to pass arguments to a function using the C CDECL calling convention.

### **Be careful not to mess up your stack.**

In this preliminary task, write a function starting with the global label `main` in assembly language. It should:

- Print `argc` in decimal format to `stdout` using `printf`.
- Print `argv[i]` to `stdout` using `puts`, for all `i` from `0` to `argc - 1`.

Write a `makefile` to compile the assembly code and link the resulting object file with the C standard library:

```
gcc myfile.o
```

This makefile will be useful throughout the lab.

---

## **The Lab Assignment**

### **Multi-Precision Integer Input-Output and Adder**

The lab work is partitioned into parts, suggesting the order of implementation and testing. Nevertheless, you are supposed to submit a single program, written entirely in assembly language, that ties everything together as stated in Part 4.

---

---

# Part 1: Structs and Multi-Precision Integer Hexadecimal Printing and Reading

Read the provided material about the difference between little endian and big endian: **little vs. big endian**.

## Part 1.A: Printing a Multi-Precision Integer

Implement:

```
print_multi(struct multi *p)
```

The function receives a pointer to:

```
struct multi {  
    unsigned char size;  
    unsigned char num[];  
};
```

where **size** is the number of bytes in the **num** array, always greater than 0, and **num** is a multi-precision unsigned integer in little endian.

The function should print the entire number in hexadecimal by calling:

```
printf("%02hhx")
```

once for every byte in the array. If the number contains leading zeros, you may remove them in the output, but this is not required.

**Warning:** C library functions do not maintain the value of all your registers.

Test this by initializing a global struct and calling **print\_multi** from **main** with a pointer to **x\_struct**:

```
x_struct: db 5  
x_num:    db 0xaa, 1, 2, 0x44, 0x4f
```

---

The output should be, with a linefeed at the end:

```
4f440201aa
```

## Part 1.B: Reading a Multi-Precision Integer

After you implement and test the printing function, implement `getmulti`. This function reads a line from `stdin` using `fgets`, containing only a sequence of hexadecimal digits, and stores it in the `multi` structure described above. Use your printing function to verify that the input is stored correctly.

You may assume:

- The input contains no leading zeros.
- The input line contains fewer than 500 characters.
- Your code will be simpler if you process hexadecimal characters in pairs.

**Think:** How can you very simply make sure you always process an even number of hex digits?

---

## Part 2: Addition of Multi-Precision Integers

### Overview

Implement:

```
struct multi *add_multi(struct multi *p, struct multi *q);
```

The function should add two numbers represented as `multi` structs and create a third number represented the same way.

---

---

The addition is performed byte by byte between the two arrays while maintaining the carry between additions. The result should be placed in a newly allocated struct with an array size of `1 + max(len1, len2)`, unless `max(len1, len2) = 255`. In that case, allocate only 255 bytes for the array, ignoring any possible overflow caused by a carry in byte 255.

## Input

Two `multi` structs containing `array1` and `array2`, of sizes `len1` and `len2` respectively, defined as `x_struct` and `y_struct` in the example below:

```
x_struct: db 5
x_num:    db 0xaa, 1, 2, 0x44, 0x4f

y_struct: db 6
y_num:    db 0xaa, 1, 2, 3, 0x44, 0x4f
```

## Output

Without loss of generality, assume that `len1 > len2`.

```
max_len = max(len1, len2) = len1
min_len = min(len1, len2) = len2
```

The function returns a new `multi` struct, dynamically allocated using `malloc`, containing `result_array`, with size `max_len + 1`, such that:

```
result_array[i] = array1[i] + array2[i] + cy    for 0 <= i < min_len
result_array[i] = array1[i] + cy                for min_len <= i < max_len
result_array[max_len] = cy
```

`cy` is the carry from the previous addition.

The result of adding the example numbers, when printed, should be:

```
4f9347040354
```

---

Leading zeros are allowed.

## Part 2.A: Get MaxMin

Implement this assembly-language function not in the C calling convention.

Given pointers to number structures in `eax` and `ebx`, return:

- In `eax`: the pointer to the structure with the higher length array.
- In `ebx`: the pointer to the other structure.

## Part 2.B: add\_multi Implementation

Use the `MaxMin` function and `print_multi` to implement and test the element-wise addition. Print each number to be added and the result, each on its own line, to `stdout`.

Test your function by defining appropriate initialized number structs and printing the resulting array.

---

## Part 3: Pseudo-Random Number Generator (PRNG)

Implement a function named `rand_num` that uses basic assembly instructions to generate a random number using a linear-feedback shift register. See the **LFSR in Wikipedia** reference.

The function uses:

- A global initialized, non-zero, unsigned 16-bit `STATE` variable.
- A constant `MASK` variable.
- The mask for the Fibonacci LFSR for 16 bits.

---

Each pseudo-random operation does the following:

- I Use `MASK` to get only the relevant bits of the `STATE` variable.
- II Compute the parity of those relevant bits. You are recommended, but not required, to use the parity flag.
- III Shift the bits of the non-masked `STATE` variable one position to the right, with the MSB determined by the parity computed in the previous step.

First, test your function by printing some generated pseudo-random numbers in hexadecimal using `printf`.

Once that works, write a function `PRmulti`, which uses the PRNG to create a pseudo-random multi-precision integer:

- The first 8 bits generated by the PRNG determine the length `n`, in bytes, of the number.
- If `n` is zero, generate a new random byte instead.
- Then `8 * n` PRNG bits determine the actual value inserted into the appropriate struct.

If implemented properly, you should be able to use your `print_multi` function from Part 1.A to print the resulting numbers. Test your code thoroughly.

---

## Part 4: Putting It All Together

Your final program, `multi`, should combine all parts above.

The program should print to `stdout`:

- I The first number to be added, in hexadecimal.
- II The second number to be added, in hexadecimal.
- III Their sum.

The program then exits normally.

The source of the numbers is determined as follows:

- With no command-line arguments, the program operates on the numbers encoded by `x_struct` and `y_struct`.
- If `argv[1]` is `-I`, the program operates on numbers obtained from `stdin`, one number per line, as in Part 1.B.
- If `argv[1]` is `-R`, the program operates on numbers obtained from the pseudo-random number generator, as in Part 3.

## Example: Default Run

```
$ ./multi
4f440201aa
4f44030201aa
4f9347040354
```

## Example: Running with `-I`

In the examples below, the first two lines are input lines. You may have an extra leading zero in each number's output.

[illegible]

```
$ ./multi -I
2
1
2
1
3
```

## Example: Running with `-R`



---

When running with the `-R` flag, the result may vary depending on the seed and on exactly how the 16-bit pseudo-random output is used. The numbers may be very long, as in the following example:

```
$ ./multi -R
5588c47a8814c200cf54a8e1dc23036edd0b196e7d5a8510c1698fdeba56294dd970a69c59
249962913d80e3bde487a991057c8a1d15d26e45a6ecb0303c480a19c61360f0eac2c039db
49a247b74ac3a0be451ecc8d7f1378196e
7f933c689717b3b90785688ffe379eb82f8faecb679a4bebaa78018761582c968fb622e3bd
f4414d5934d765abe8b3597cc218ae337dbae915580733f71b66af6c0f19a83baa6842885
a8f1922439bbeb1d848e25d2768070d039ee84b539a83aa6060e0b4031e24d9588cd7d52c
56e11cd58c8f9e908d27d027304cb1d85824b9ef59ace2b563897c6afe77b4632112508b8d
b7ca9717a37fe34d5924118163d918b6ca3f79990a194ef092147297b7e22579395b85304c
a11ebcec8844b61a1fd454b0189ee074f591c59ac3189e7851e3bd6c645b759d29fdc6ebe2
f55994963f319c
7f933c689717b3b90785688ffe379eb82f8faecb679a4bebaa78018761582c968fb622e3bd
f4414d5934d765abe8b3597cc218ae337dbae915580733f71b66af6c0f19a83baa6842885
a8f1922439bbeb1d848e25d2768070d039ee84b539a83aa6060e0b4031e24d9588cd7d52c
56e11cd58c8f9e908d27d027304cb1d85824b9ef59ace2b563897cc0873c2eeb35d4515ae2
60ac733aa6eec05872928edbe8e9da205a1e33ef3367286138b0cbbc5144b6b6ba3f4314d4
4aafc2691261cbec8e19fb9cc8cf1cbcfab8bae2409893b121d98b606a32ce7ed9e853101
cle713a9b74b0a
```

---

## Submission

Submit a zip file named:

```
[your_id].zip
```

The zip file must contain:

- `multi.s`: a single assembly-language source file.
- `makefile`: compiles and links the program into an executable named `multi`.

Compile and link with:

---

```
nasm -f elf32 multi.s -o multi.o  
gcc -m32 multi.o -o multi
```

The submitted code is the completed Part 4 program, containing all previous parts.